

Asterisk 기반 콜센터 CTI 구축

실전 개발용 상세 설계서 v1.0

대상: 오픈소스 IP-PBX Asterisk 를 이용한 콜센터 CTI 구축 프로젝트

문서 목적: 개발 착수, 외주 전달, 내부 기술 검토, 단계별 구현 기준으로 활용

핵심 원칙

- Asterisk 는 통화 제어에 집중하고, 업무 로직은 CTI Middleware 에서 처리한다.
- 상담원 화면은 PBX 내부 상태를 직접 해석하지 않고, Middleware 가 정규화한 이벤트를 사용한다.
- CRM, 주문, 고객, 상담 이력, 녹취 메타데이터는 별도 업무 DB 에 저장한다.
- AMI 는 기본 이벤트 수집, ARI 는 특수 콜 제어에 한정해 단계적으로 적용한다.

1. 프로젝트 범위와 목표

본 시스템은 대표번호 인입, IVR, Queue, 상담원 분배, 고객 팝업, 통화 메모, 녹취, 통계, 관리자 모니터링까지 포함하는 콜센터 CTI 플랫폼을 구축하는 것을 목표로 한다. 단순 PBX 설치가 아니라 전화 이벤트를 업무 시스템과 연결하는 통합 플랫폼 구축이 범위다.

1.1 구축 목표

- 인바운드 대표번호 수신과 상담원 분배 자동화
- 고객 전화번호 기준 CRM/고객 DB 자동 조회
- 상담원 상태 관리 및 후처리 코드 표준화
- 실시간 상태판, Queue 현황, 기본 운영 통계 제공
- 녹취 파일과 콜 메타데이터 통합 관리
- 향후 STT, AI 요약, 품질평가 기능 확장 가능 구조 확보

1.2 범위 포함

- Asterisk 22 LTS 기반 PBX
- PJSIP 내선/트렁크 구성
- AMI 이벤트 수집 기반 CTI Middleware
- 상담원 웹 CTI, 관리자 콘솔
- PostgreSQL 업무 DB
- 녹취 및 이벤트 로그 저장

1.3 범위 제외

- 예측 발신기(Predictive Dialer)의 고급 알고리즘
- WFM, 인사, 급여 등 비콜센터 백오피스
- 통신사 회선 품질 자체 보장
- AI 엔진 자체 학습 플랫폼

2. 상위 아키텍처

아키텍처는 PBX 계층, CTI Middleware 계층, 웹 UI 계층, 업무/통계 DB 계층으로 분리한다. 유지보수성과 장애 격리를 위해 각 계층의 책임을 명확히 나눈다.

계층	주요 컴포넌트	책임
통화 제어	Asterisk, PJSIP, Queue, IVR, MixMonitor	호 수립, 라우팅, 브리지, 녹취
이벤트/업무	CTI Middleware, Event Consumer, REST API, WebSocket	AMI 이벤트 수집, 세션 상태 관리, CRM 연동, UI 전달
사용자 화면	상담원 CTI, 관리자 콘솔	통화 제어 요청, 상태 조회, 메모 입력, 모니터링
데이터	PostgreSQL, 파일 스토리지, 로그 저장소	콜 이력, 상담 이력, 녹취 메타데이터, 통계

2.1 권장 물리 구성

- PBX 서버: Asterisk 전용 서버. 실시간 RTP 품질을 위해 업무 API와 분리 권장
- CTI/App 서버: Node.js 또는 Python 기반 Middleware와 웹 서비스
- DB 서버: PostgreSQL. 운영 규모가 커지면 별도 분리
- 파일 스토리지: 녹취 저장 전용 NAS 또는 Object Storage
- 리버스 프록시: Nginx. HTTPS와 WebSocket 처리

2.2 모듈 경계

- Asterisk 는 고객/주문 비즈니스 규칙을 알지 못한다.
- Middleware 는 PBX 이벤트를 정규화하여 WebSocket 과 REST API 로 노출한다.
- 상담원 화면은 반드시 내부 상태머신을 재구성하지 않고 서버의 세션 상태를 따른다.

3. 핵심 콜 시나리오

3.1 인바운드 기본 시나리오

1. 고객이 대표번호로 전화한다.
2. 통신사 SIP Trunk 를 통해 Asterisk 로 인입된다.
3. Dialplan 이 업무시간, DID, 캠페인 규칙을 평가한다.
4. 필요 시 IVR 안내 후 Queue 에 진입한다.
5. Queue 정책에 따라 가용 상담원에게 전화가 분배된다.
6. 상담원 전화가 올리면 AMI 이벤트를 Middleware 가 수신한다.
7. Middleware 는 ANI 기준으로 고객 정보를 조회하고 상담원 화면에 팝업을 보낸다.
8. 통화 종료 후 상담원은 후처리 코드와 메모를 저장한다.
9. 콜 이력, 녹취 정보, 상담 메모를 DB 에 확정 저장한다.

3.2 미응답/포기호

- Queue 대기 중 끊기면 abandoned 상태로 저장
- 상담원 미응답 후 재분배 시 동일 linkedid 기준으로 세션 유지
- 임계시간 이상 대기 시 콜백 접수 메뉴 또는 음성안내로 분기 가능

3.3 상담원 전환

- 블라인드 전환: 목적 내선으로 즉시 전달
- 상담 전 협의 전환: attended transfer 사용
- 전환 이력은 원 통화 linkedid 에 묶어 저장

3.4 아웃바운드 클릭투콜

- 상담원 화면에서 고객 번호 선택
- Middleware 가 originate 요청
- 상담원 내선이 먼저 벨 수신
- 상담원 응답 후 고객 번호로 외부 발신
- 성공 시 동일 화면에서 메모 및 결과 등록

4. Asterisk 설계

4.1 버전 및 주요 모듈

구분	선택	비고
Asterisk 버전	22 LTS	신규 구축 권장
SIP 스택	PJSIP	chan_sip 신규 사용 지양
Queue	app_queue	콜센터 핵심
녹취	MixMonitor	브리지 단위 녹취
이벤트 연동	AMI	기본 CTI 이벤트 수집
고급 제어	ARI	2 차 단계 선택 적용

4.2 권장 파일 구조

설정 파일은 역할별로 분리해 운영한다. 한 파일에 모든 컨텍스트를 몰아넣지 않는다.

- /etc/asterisk/pjsip.conf - 트렁크, 엔드포인트, AOR, 인증
- /etc/asterisk/extensions.conf - 메인 dialplan 진입점
- /etc/asterisk/extensions_inbound.conf - 인바운드 라우팅
- /etc/asterisk/extensions_queue.conf - Queue 진입/이탈 로직
- /etc/asterisk/extensions_agent.conf - 상담원 관련 제어
- /etc/asterisk/queues.conf - 대기열 정책
- /etc/asterisk/manager.conf - AMI 계정 및 ACL
- /etc/asterisk/ari.conf - ARI 계정 및 앱

4.3 Dialplan 원칙

- inbound, ivr, queue, agent, outbound 컨텍스트를 분리한다.
- DB 조회나 업무 판단을 Dialplan에 과도하게 넣지 않는다.
- 외부 업무 규칙은 AGI/FastAGI 또는 Middleware API 호출로 분리한다.
- linkedid를 기준으로 하나의 비즈니스 세션을 추적한다.

4.4 Queue 정책 권고

항목	권고값	설명
strategy	leastrecent 또는 fewestcalls	상담원 편중 완화
timeout	15-20 초	한 상담원 울림 시간
retry	2-3 초	다음 상담원 전 분기 간격
wrapuptime	20-60 초	후처리 시간
autopause	yes	미응답 상담원 자동 제외
setinterfacevar	yes	이벤트 추적용 변수

5. CTI Middleware 설계

Middleware는 PBX와 웹/업무 시스템 사이의 핵심 허브다. Asterisk 이벤트를 세션 모델로 재구성하고, 고객 조회, 상태 저장, 웹 푸시, 외부 API 연동을 수행한다.

5.1 권장 내부 모듈

모듈	역할
AMI Connector	Asterisk AMI 접속, 이벤트 수신, 재접속
Session Engine	채널/브리지/Queue 이벤트를 콜 세션으로 정규화
Agent State Service	상담원 상태, 로그인, 후처리, pause 코드 관리
CRM Adapter	전화번호 기반 고객 조회, 고객 상세/주문 이력 호출
Call Command API	originate, hold, transfer, hangup 요청 처리
WebSocket Gateway	상담원/관리자 화면으로 실시간 이벤트 송신
Persistence Layer	콜 이력, 이벤트 로그, 메모, 녹취 메타데이터 저장

5.2 세션 상태 머신

AMI 이벤트는 개별 채널 기준이므로 그대로 UI에 보내면 중복과 혼선이 발생한다. linkedid를 기준으로 세션을 만들고, 세션 상태를 아래와 같이 관리한다.

세션 상태	설명
NEW	인입 감지 직후, 상담원 미할당
IVR	IVR/안내 구간
QUEUED	Queue 대기 중
RINGING_AGENT	상담원에게 분배되어 벨 울림
TALKING	상담원 통화 중

HOLD	보류 상태
TRANSFERRING	전환 진행 중
AFTER_CALL_WORK	통화 종료 후 후처리 중
ENDED	세션 종료 및 이력 저장 완료

5.3 정합성 처리 원칙

- Event 순서 역전 가능성을 고려하여 idempotent 처리
- Uniqueid 보다 linkedid 중심 집계
- AMI 재접속 후 주기적 상태 재동기화
- 장시간 남은 유령 세션은 타임아웃 정리
- DB 저장은 이벤트 원본 로그와 정규화 세션을 분리

6. AMI 이벤트 처리 설계

6.1 중요 이벤트

AMI Event	용도
Newchannel	새 채널 생성 감지
Newstate	Ringin, Up 등 채널 상태 변화
DialBegin / DialEnd	상담원 또는 외부 대상 발신 시도/결과
BridgeEnter / BridgeLeave	실제 통화 연결 판단
QueueCallerJoin / QueueCallerLeave	대기열 입/이탈
AgentCalled / AgentConnect / AgentComplete	Queue 상담원 호출, 연결, 완료
Hangup	채널 종료
VarSet	커스텀 채널 변수 감시
Cdr / CEL 연동	이력 보강

6.2 예시 이벤트 흐름

인바운드 Queue 콜 기준으로 예상 흐름은 Newchannel -> QueueCallerJoin -> AgentCalled -> AgentConnect -> BridgeEnter -> Hangup -> AgentComplete 순서다. 실제 운영에서는 전환, 재시도, 중간 끊김으로 인해 분기가 다수 발생하므로 세션 엔진이 필수다.

6.3 이벤트 저장 전략

- raw_ami_events: 원본 이벤트 전문 JSON 저장
- call_sessions: linkedid 단위 현재 세션 스냅샷
- call_legs: 개별 채널/구간 단위 저장
- agent_presence_history: 상담원 상태 변경 이력
- queue_fact: Queue 진입, 응답, 포기, 대기시간

7. API 설계

7.1 API 원칙

- REST API는 업무 CRUD, WebSocket 은 실시간 푸시
- 콜 제어 API는 권한 체크 후 서버가 PBX 로 위임
- 상담원 UI는 PBX 에 직접 접속하지 않는다.

7.2 핵심 API 목록

Method	Path	설명
POST	/api/auth/login	상담원 로그인

GET	/api/me/session	현재 상담원 세션 정보
POST	/api/agents/{id}/status	상담원 상태 변경
GET	/api/calls/active	현재 활성 콜 목록
GET	/api/calls/{callId}	콜 상세
POST	/api/calls/{callId}/memo	상담 메모 저장
POST	/api/calls/originate	클릭투콜 발신
POST	/api/calls/{callId}/transfer	전환 요청
GET	/api/customers/search?phone=	전화번호 기반 고객 조회
GET	/api/queues/summary	Queue 요약 정보
GET	/api/admin/dashboard	관리자 대시보드

7.3 WebSocket 이벤트 예시

- call.created
- call.updated
- call.ended
- agent.status.changed
- queue.summary.updated
- screenpop.customer

8. DB 설계

8.1 핵심 엔티티

테이블	주요 용도	핵심 키
agents	상담원 마스터	agent_id, extension
agent_status_history	상담원 상태 이력	status_history_id, agent_id
customers	고객 기본 정보	customer_id, phone_number
call_sessions	콜 세션 헤더	call_id, linkedid
call_legs	개별 호 구간	leg_id, uniqueid, linkedid
call_recordings	녹취 메타데이터	recording_id, call_id
call_memos	상담 메모	memo_id, call_id, agent_id
queue_events	Queue 이벤트 팩트	queue_event_id, linkedid
raw_ami_events	원본 이벤트 로그	event_id

8.2 call_sessions 권장 컬럼

- call_id UUID PK
- linkedid VARCHAR(32) UNIQUE
- direction inbound/outbound/internal
- ani, dnis, queue_name, campaign_code
- customer_id nullable
- primary_agent_id nullable
- session_status
- started_at, queued_at, answered_at, ended_at
- wait_seconds, talk_seconds, acw_seconds
- result_code, abandon_flag, transfer_flag

8.3 인덱스 권장

- call_sessions(linkedid), call_sessions(started_at), call_sessions(primary_agent_id, started_at)
- customers(phone_number)
- raw_ami_events(linkedid, event_time)
- queue_events(queue_name, event_time)

8.4 샘플 SQL DDL

```
CREATE TABLE call_sessions (  
    call_id UUID PRIMARY KEY,  
    linkedid VARCHAR(32) NOT NULL UNIQUE,  
    direction VARCHAR(16) NOT NULL,  
    ani VARCHAR(32),  
    dnis VARCHAR(32),  
    queue_name VARCHAR(64),  
    customer_id UUID NULL,  
    primary_agent_id UUID NULL,  
    session_status VARCHAR(32) NOT NULL,  
    started_at TIMESTAMPTZ NOT NULL,  
    queued_at TIMESTAMPTZ NULL,  
    answered_at TIMESTAMPTZ NULL,  
    ended_at TIMESTAMPTZ NULL,  
    wait_seconds INTEGER DEFAULT 0,  
    talk_seconds INTEGER DEFAULT 0,  
    acw_seconds INTEGER DEFAULT 0,  
    result_code VARCHAR(32),  
    abandon_flag BOOLEAN DEFAULT FALSE,  
    transfer_flag BOOLEAN DEFAULT FALSE  
);
```

9. 상담원 웹 CTI 설계

9.1 화면 구성

- 상단 바: 로그인 상담원, 상태, 내선, 오늘 통계
- 좌측: 상태 변경, 재콜 목록, 개인 큐
- 중앙: 현재 통화 카드, 고객 정보, 주문/이력 요약
- 우측: 메모 입력, 후처리 코드, 전환/보류/재다이얼
- 하단: 최근 통화 이력, 시스템 알림

9.2 UX 원칙

- 인입 1 초 내 고객 기본 정보 팝업
- 화면에서 필요한 조작을 3 클릭 내로 제한
- 통화 종료 즉시 후처리 카드 자동 노출
- 상태 전환 시 이유 코드 선택 지원
- 오류 시 PBX 원문 노출 대신 사용자 친화 메시지 제공

10. 관리자 콘솔 설계

10.1 대시보드

- 실시간 대기 콜 수, 응답률, 포기율, 평균 대기시간
- 팀별 상담원 로그인/통화/이석 현황
- Queue 별 SLA 초과 건수
- 시간대별 유입량 차트

10.2 검색/감사

- 전화번호, 상담원, 일자, 결과코드 기준 콜 검색
- 녹취 재생 및 메모 조회
- 전환/민원/장기대기 등 특이콜 필터
- 권한별 마스킹과 접근 제어

11. 보안 및 권한

11.1 네트워크

- Asterisk AMI 는 내부망 또는 VPN 에서만 허용
- manager.conf 에 permit/deny ACL 설정
- Web 서비스는 HTTPS 만 허용
- SIP 트렁크는 통신사 IP 화이트리스트 적용

11.2 애플리케이션

- JWT 또는 세션 기반 인증
- 상담원/슈퍼바이저/관리자 역할 분리
- 고객 전화번호, 카드정보 등 민감 정보는 마스킹
- 감사 로그 저장: 로그인, 조회, 다운로드, 녹취 접근

12. 장애 대응과 운영

12.1 장애 시나리오

- AMI 연결 끊김: 자동 재접속, 상태 재동기화, 운영 알림
- DB 지연: 이벤트 큐 적재 후 비동기 저장
- Asterisk 재시작: 트렁크 재등록 확인, 헬스체크, Queue 멤버 재동기화
- 스토리지 장애: 녹취 실패 감지 및 경보

12.2 모니터링

- PBX: 채널 수, RTP 지연, SIP 등록, 트렁크 상태
- App: AMI 연결 상태, WebSocket 접속 수, API 응답시간
- DB: 커넥션, 슬로우쿼리, 디스크 사용량
- 스토리지: 녹취 생성 성공률, 용량 임계치

13. 배포 및 환경

13.1 환경 구분

- DEV: 단일 서버, 테스트 트렁크, 샘플 데이터
- STG: 운영 유사 구성, 실제 부하 전 테스트
- PRD: PBX/App/DB 분리, 백업 및 모니터링 포함

13.2 CI/CD

- 애플리케이션은 Git 기반 배포
- DB 마이그레이션 스크립트 버전 관리
- Asterisk 설정은 Git 으로 관리 후 점진 반영
- 운영 반영 전 dialplan syntax 검증

14. 단계별 구현 로드맵

단계	기간	주요 산출물
1 단계 PoC	2 주	트렁크 연결, Queue, AMI 수신, 기본 화면

2 단계 MVP	4 주	상담원 CTI, 고객조회, 메모, 녹취, 관리자 기본판
3 단계 안정화	2 주	정합성 보강, 통계, 장애 알림, 권한
4 단계 고도화	4 주	ARI 특수 제어, 콜백, STT/AI 연동

14.1 PoC 완료 기준

- 대표번호 인입과 Queue 연결 성공
- AMI 이벤트 기반 고객 팝업 동작
- 콜 이력과 녹취 메타데이터 저장
- 상담원 상태 변경과 대시보드 반영

14.2 MVP 완료 기준

- 상담원 20-50 석 기준 파일럿 운영 가능
- 기본 리포트와 관리자 검색 제공
- 장애 알림과 운영 매뉴얼 정리

15. Asterisk 설정 초안 예시

15.1 pjsip.conf 개념 예시

```
[trunk-provider]
type=endpoint
transport=transport-udp
context=inbound-main
disallow=all
allow=alaw,ulaw
aors=trunk-provider-aor
outbound_auth=trunk-provider-auth
```

```
[1001]
type=endpoint
context=agent-phone
disallow=all
allow=alaw,ulaw
auth=1001-auth
aors=1001-aor
callerid=Agent1001 <1001>
```

15.2 extensions.conf 개념 예시

```
[inbound-main]
exten => 07012345678,1,NoOp(Inbound DID)
same => n,Set(__ENTRY_DID=${EXTEN})
same => n,Goto(queue-sales,s,1)

[queue-sales]
exten => s,1,Answer()
same => n,Set(__CALL_LINKEDID=${CHANNEL(linkedid)})
same => n,Queue(sales,tT,,,45)
same => n,Hangup()
```

16. 테스트 전략

16.1 기능 테스트

- 인입, IVR, Queue, 응답, 종료, 녹취 생성
- 미응답, 상담원 이석, 자동 pause, 재분배
- 전환, 보류, 외부 발신, 재콜 등록

16.2 비기능 테스트

- 동시 통화 부하 테스트
- 이벤트 폭주 시 세션 정합성 확인
- Asterisk 재시작 후 복구 테스트
- DB 장애/지연 상황에서 유실 여부 점검

17. 향후 확장

- STT: 녹취 파일을 STT 파이프라인에 연계
- AI 요약: 통화 종료 후 요약 및 민원 태깅
- QA 평가: 금칙어, 스크립트 준수율 분석
- 멀티채널: 채팅/카카오톡/이메일 통합 에이전트 화면

18. 결론

실전 개발에서는 PBX 기능 구현보다 이벤트 정합성과 운영 편의성이 더 중요하다. 본 설계서는 Asterisk를 통화 엔진으로 사용하고, CTI Middleware를 중심으로 업무 로직을 분리하는 구조를 채택했다. 이 방식은 초기 구축 난이도와 운영 안정성의 균형이 가장 좋으며, 향후 STT/AI 기능 확장에도 유리하다.

개발 시작 시 우선순위는 1) 트렁크/Queue/녹취 PoC, 2) AMI 세션 엔진, 3) 상담원 CTI, 4) 관리자 모니터링, 5) 통계/안정화 순으로 진행하는 것을 권장한다.